

Parameter Update in Neural Networks

ReLU Output and Softmax Output

CSE427: Machine Learning

Network Structure

We consider a neural network with:

Input layer: X_1, X_2

Hidden layer: Y_3, Y_4

Output layer: Y_5, Y_6, Y_7

The hidden nodes are:

$$Y_3 = \text{ReLU}(W_{13}X_1 + W_{23}X_2 + b_3)$$

$$Y_4 = \text{ReLU}(W_{14}X_1 + W_{24}X_2 + b_4)$$

The output nodes may use either:

ReLU output

or:

Softmax output

What Are We Trying to Learn?

The network contains many parameters:

$$W_{13}, W_{23}, W_{14}, W_{24}$$

$$W_{35}, W_{45}, W_{36}, W_{46}, W_{37}, W_{47}$$

$$b_3, b_4, b_5, b_6, b_7$$

Training means updating these parameters so that prediction becomes closer to target output.

$$\text{New parameter} = \text{Old parameter} - \eta \frac{\partial \text{Loss}}{\partial \text{parameter}}$$

where:

$$\eta = \text{learning rate}$$

Forward Pass: Hidden Layer

First compute weighted sums before activation:

$$z_3 = W_{13}X_1 + W_{23}X_2 + b_3$$

$$z_4 = W_{14}X_1 + W_{24}X_2 + b_4$$

Then apply ReLU:

$$Y_3 = \text{ReLU}(z_3)$$

$$Y_4 = \text{ReLU}(z_4)$$

where:

$$\text{ReLU}(z) = \begin{cases} z, & z > 0 \\ 0, & z \leq 0 \end{cases}$$

$$\text{ReLU}'(z) = \begin{cases} 1, & z > 0 \\ 0, & z \leq 0 \end{cases}$$

Forward Pass: Output Layer

The output pre-activation values are:

$$z_5 = W_{35} Y_3 + W_{45} Y_4 + b_5$$

$$z_6 = W_{36} Y_3 + W_{46} Y_4 + b_6$$

$$z_7 = W_{37} Y_3 + W_{47} Y_4 + b_7$$

For ReLU output:

$$Y_5 = \text{ReLU}(z_5)$$

$$Y_6 = \text{ReLU}(z_6)$$

$$Y_7 = \text{ReLU}(z_7)$$

For softmax output:

$$P_5 = \frac{e^{z_5}}{e^{z_5} + e^{z_6} + e^{z_7}}$$

$$P_6 = \frac{e^{z_6}}{e^{z_5} + e^{z_6} + e^{z_7}}$$

$$P_7 = \frac{e^{z_7}}{e^{z_5} + e^{z_6} + e^{z_7}}$$

Part A: ReLU Output with Squared Error

Suppose the observed outputs are:

$$Ob_5, \quad Ob_6, \quad Ob_7$$

Predicted outputs are:

$$Y_5, \quad Y_6, \quad Y_7$$

Use squared error loss:

$$Loss = \frac{1}{2} [(Ob_5 - Y_5)^2 + (Ob_6 - Y_6)^2 + (Ob_7 - Y_7)^2]$$

Define output errors:

$$Err_5 = Ob_5 - Y_5$$

$$Err_6 = Ob_6 - Y_6$$

$$Err_7 = Ob_7 - Y_7$$

Derivative of Loss with Respect to Output

For output node 5:

$$Loss_5 = \frac{1}{2}(Ob_5 - Y_5)^2$$

Differentiate:

$$\begin{aligned}\frac{\partial Loss}{\partial Y_5} &= \frac{1}{2} \cdot 2(Ob_5 - Y_5)(-1) \\ &= -(Ob_5 - Y_5) \\ &= -Err_5\end{aligned}$$

Similarly:

$$\frac{\partial Loss}{\partial Y_6} = -Err_6$$

$$\frac{\partial Loss}{\partial Y_7} = -Err_7$$

$$\boxed{\frac{\partial Loss}{\partial Y_j} = -Err_j}$$

Output Layer Delta for ReLU Output

For output node 5:

$$Y_5 = \text{ReLU}(z_5)$$

where:

$$z_5 = W_{35} Y_3 + W_{45} Y_4 + b_5$$

Using chain rule:

$$\begin{aligned} \frac{\partial \text{Loss}}{\partial z_5} &= \frac{\partial \text{Loss}}{\partial Y_5} \frac{\partial Y_5}{\partial z_5} \\ &= (-\text{Err}_5) \text{ReLU}'(z_5) \end{aligned}$$

Define:

$$\delta_5 = \text{Err}_5 \text{ReLU}'(z_5)$$

Then:

$$\frac{\partial \text{Loss}}{\partial z_5} = -\delta_5$$

Similarly:

$$\delta_6 = \text{Err}_6 \text{ReLU}'(z_6)$$

$$\delta_7 = \text{Err}_7 \text{ReLU}'(z_7)$$

Update W_{35} Step by Step

We want:

$$\frac{\partial \text{Loss}}{\partial W_{35}}$$

Use chain rule:

$$\frac{\partial \text{Loss}}{\partial W_{35}} = \frac{\partial \text{Loss}}{\partial Y_5} \frac{\partial Y_5}{\partial z_5} \frac{\partial z_5}{\partial W_{35}}$$

Now:

$$\frac{\partial \text{Loss}}{\partial Y_5} = -\text{Err}_5$$

$$\frac{\partial Y_5}{\partial z_5} = \text{ReLU}'(z_5)$$

$$\frac{\partial z_5}{\partial W_{35}} = Y_3$$

So:

$$\begin{aligned} \frac{\partial \text{Loss}}{\partial W_{35}} &= -\text{Err}_5 \text{ReLU}'(z_5) Y_3 \\ &= -Y_3 \delta_5 \end{aligned}$$

Update:

ReLU Output Layer Updates

For node 5:

$$W_{35}^{new} = W_{35} + \eta Y_3 \delta_5$$

$$W_{45}^{new} = W_{45} + \eta Y_4 \delta_5$$

$$b_5^{new} = b_5 + \eta \delta_5$$

For node 6:

$$W_{36}^{new} = W_{36} + \eta Y_3 \delta_6$$

$$W_{46}^{new} = W_{46} + \eta Y_4 \delta_6$$

$$b_6^{new} = b_6 + \eta \delta_6$$

For node 7:

$$W_{37}^{new} = W_{37} + \eta Y_3 \delta_7$$

$$W_{47}^{new} = W_{47} + \eta Y_4 \delta_7$$

$$b_7^{new} = b_7 + \eta \delta_7$$

Why Hidden Layer Is Harder

Output weights are easier because each output node directly affects the loss.
But hidden node Y_3 affects all output nodes:

$$Y_3 \rightarrow Y_5$$

$$Y_3 \rightarrow Y_6$$

$$Y_3 \rightarrow Y_7$$

So the loss reaches Y_3 through three paths.
Therefore:

$$\frac{\partial \text{Loss}}{\partial Y_3} = \text{contribution from node 5} + \text{contribution from node 6} + \text{contribution from node 7}$$

A hidden neuron receives error from all neurons it connects to.

Hidden Error for Y_3

Since:

$$z_5 = W_{35} Y_3 + W_{45} Y_4 + b_5$$

$$z_6 = W_{36} Y_3 + W_{46} Y_4 + b_6$$

$$z_7 = W_{37} Y_3 + W_{47} Y_4 + b_7$$

The hidden node Y_3 contributes through:

$$W_{35}, \quad W_{36}, \quad W_{37}$$

Using output deltas:

$$\delta_3 = (\delta_5 W_{35} + \delta_6 W_{36} + \delta_7 W_{37}) \text{ReLU}'(z_3)$$

This means:

hidden error = weighted sum of next-layer errors $\times$ activation derivative

Hidden Error for Y_4

Similarly, hidden node Y_4 connects to output nodes through:

$$W_{45}, \quad W_{46}, \quad W_{47}$$

So:

$$\delta_4 = (\delta_5 W_{45} + \delta_6 W_{46} + \delta_7 W_{47}) \text{ReLU}'(z_4)$$

Interpretation:

$$\delta_5 W_{45}$$

is the part of output node 5 error that returns to hidden node 4.

$$\delta_6 W_{46}$$

is the part of output node 6 error that returns to hidden node 4.

$$\delta_7 W_{47}$$

is the part of output node 7 error that returns to hidden node 4.

Hidden Layer Parameter Updates

For hidden node 3:

$$z_3 = W_{13}X_1 + W_{23}X_2 + b_3$$

$$Y_3 = \text{ReLU}(z_3)$$

Updates:

$$W_{13}^{\text{new}} = W_{13} + \eta X_1 \delta_3$$

$$W_{23}^{\text{new}} = W_{23} + \eta X_2 \delta_3$$

$$b_3^{\text{new}} = b_3 + \eta \delta_3$$

For hidden node 4:

$$W_{14}^{\text{new}} = W_{14} + \eta X_1 \delta_4$$

$$W_{24}^{\text{new}} = W_{24} + \eta X_2 \delta_4$$

$$b_4^{\text{new}} = b_4 + \eta \delta_4$$

Part B: Softmax Output

Now suppose the output layer does not use ReLU.
Instead, it uses softmax.

First compute logits:

$$Y_5 = W_{35} Y_3 + W_{45} Y_4 + b_5$$

$$Y_6 = W_{36} Y_3 + W_{46} Y_4 + b_6$$

$$Y_7 = W_{37} Y_3 + W_{47} Y_4 + b_7$$

Then softmax gives probabilities:

$$P_5 = \frac{e^{Y_5}}{e^{Y_5} + e^{Y_6} + e^{Y_7}}$$

$$P_6 = \frac{e^{Y_6}}{e^{Y_5} + e^{Y_6} + e^{Y_7}}$$

$$P_7 = \frac{e^{Y_7}}{e^{Y_5} + e^{Y_6} + e^{Y_7}}$$

Softmax Output Meaning

Softmax converts raw scores into probabilities.
If:

$$P_5 = 0.70, \quad P_6 = 0.20, \quad P_7 = 0.10$$

then the model is saying:

Class 5 probability = 70%

Class 6 probability = 20%

Class 7 probability = 10%

Softmax probabilities always sum to 1:

$$P_5 + P_6 + P_7 = 1$$

Softmax is used for multi-class classification.

Softmax with Cross-Entropy Loss

Suppose the correct class is class 5.

Then the target is:

$$t_5 = 1, \quad t_6 = 0, \quad t_7 = 0$$

The loss is:

$$Loss = -\log(P_5)$$

Because class 5 is correct, we want P_5 to become large.

If P_5 is small:

$$-\log(P_5)$$

becomes large.

If P_5 is close to 1:

$$-\log(P_5)$$

becomes close to 0.

Cross-entropy punishes low probability for the correct class.

Important Softmax Derivatives

For correct class 5:

$$Loss = -\log(P_5)$$

The important result is:

$$\frac{\partial Loss}{\partial Y_5} = P_5 - 1$$

$$\frac{\partial Loss}{\partial Y_6} = P_6$$

$$\frac{\partial Loss}{\partial Y_7} = P_7$$

In general:

$$\boxed{\frac{\partial Loss}{\partial Y_j} = P_j - t_j}$$

where:

$$t_j = \begin{cases} 1, & \text{if } j \text{ is correct class} \\ 0, & \text{otherwise} \end{cases}$$

Why We Need to Derive the Softmax Gradient

For softmax output, the output nodes first produce raw scores:

$$Y_5, \quad Y_6, \quad Y_7$$

These raw scores are called logits.

Softmax converts them into probabilities:

$$P_5 = \frac{e^{Y_5}}{e^{Y_5} + e^{Y_6} + e^{Y_7}}$$

$$P_6 = \frac{e^{Y_6}}{e^{Y_5} + e^{Y_6} + e^{Y_7}}$$

$$P_7 = \frac{e^{Y_7}}{e^{Y_5} + e^{Y_6} + e^{Y_7}}$$

Suppose the correct class is class 5.

Then the loss is:

$$Loss = -\log(P_5)$$

Now we need $\frac{\partial Loss}{\partial Y_5}$, $\frac{\partial Loss}{\partial Y_6}$, $\frac{\partial Loss}{\partial Y_7}$

Deriving $\frac{\partial P_5}{\partial Y_5}$

Start with:

$$P_5 = \frac{e^{Y_5}}{e^{Y_5} + e^{Y_6} + e^{Y_7}}$$

Let:

$$A = e^{Y_5}$$

$$B = e^{Y_6} + e^{Y_7}$$

Then:

$$P_5 = \frac{A}{A + B}$$

Now differentiate with respect to Y_5 .

Since:

$$\frac{dA}{dY_5} = A$$

and:

$$\frac{dB}{dY_5} = 0$$

using quotient rule:

Getting $P_5(1 - P_5)$

From the previous slide:

$$\frac{\partial P_5}{\partial Y_5} = \frac{AB}{(A+B)^2}$$

Rewrite it as:

$$\frac{\partial P_5}{\partial Y_5} = \frac{A}{A+B} \cdot \frac{B}{A+B}$$

But:

$$\frac{A}{A+B} = P_5$$

Also:

$$\frac{B}{A+B} = 1 - \frac{A}{A+B}$$

So:

$$\frac{B}{A+B} = 1 - P_5$$

Therefore:

$$\boxed{\frac{\partial P_5}{\partial Y_5} = P_5(1 - P_5)}$$

Now Derive $\frac{\partial \text{Loss}}{\partial Y_5}$

We know:

$$\text{Loss} = -\log(P_5)$$

Using chain rule:

$$\frac{\partial \text{Loss}}{\partial Y_5} = \frac{\partial \text{Loss}}{\partial P_5} \cdot \frac{\partial P_5}{\partial Y_5}$$

First:

$$\frac{\partial \text{Loss}}{\partial P_5} = -\frac{1}{P_5}$$

From the previous slide:

$$\frac{\partial P_5}{\partial Y_5} = P_5(1 - P_5)$$

So:

$$\frac{\partial \text{Loss}}{\partial Y_5} = -\frac{1}{P_5} \cdot P_5(1 - P_5)$$

Cancel P_5 :

$$= -(1 - P_5)$$

$$= -1 + P_5$$

Softmax Output Layer Updates

For class 5 output weights:

$$\frac{\partial \text{Loss}}{\partial W_{35}} = (P_5 - 1)Y_3$$

$$\frac{\partial \text{Loss}}{\partial W_{45}} = (P_5 - 1)Y_4$$

$$\frac{\partial \text{Loss}}{\partial b_5} = P_5 - 1$$

For class 6:

$$\frac{\partial \text{Loss}}{\partial W_{36}} = P_6 Y_3$$

$$\frac{\partial \text{Loss}}{\partial W_{46}} = P_6 Y_4$$

$$\frac{\partial \text{Loss}}{\partial b_6} = P_6$$

For class 7:

$$\frac{\partial \text{Loss}}{\partial W_{37}} = P_7 Y_3$$

$$\frac{\partial \text{Loss}}{\partial W_{47}} = P_7 Y_4$$

Softmax Gradient Descent Updates

Using:

$$\theta^{new} = \theta - \eta \frac{\partial Loss}{\partial \theta}$$

we get:

$$W_{35}^{new} = W_{35} - \eta Y_3 (P_5 - 1)$$

$$W_{45}^{new} = W_{45} - \eta Y_4 (P_5 - 1)$$

$$W_{36}^{new} = W_{36} - \eta Y_3 P_6$$

$$W_{46}^{new} = W_{46} - \eta Y_4 P_6$$

$$W_{37}^{new} = W_{37} - \eta Y_3 P_7$$

$$W_{47}^{new} = W_{47} - \eta Y_4 P_7$$

Bias updates:

$$b_5^{new} = b_5 - \eta (P_5 - 1)$$

$$b_6^{new} = b_6 - \eta P_6$$

$$b_7^{new} = b_7 - \eta P_7$$

Softmax Hidden Layer Error

Hidden node Y_3 affects all output logits:

$$Y_5, \quad Y_6, \quad Y_7$$

So:

$$\frac{\partial \text{Loss}}{\partial Y_3} = (P_5 - 1)W_{35} + P_6 W_{36} + P_7 W_{37}$$

Then multiply by ReLU derivative at hidden node 3:

$$\delta_3 = [(P_5 - 1)W_{35} + P_6 W_{36} + P_7 W_{37}] \text{ReLU}'(z_3)$$

For node 4:

$$\delta_4 = [(P_5 - 1)W_{45} + P_6 W_{46} + P_7 W_{47}] \text{ReLU}'(z_4)$$

Softmax Hidden Parameter Updates

For hidden node 3:

$$W_{13}^{new} = W_{13} - \eta \delta_3 X_1$$

$$W_{23}^{new} = W_{23} - \eta \delta_3 X_2$$

$$b_3^{new} = b_3 - \eta \delta_3$$

For hidden node 4:

$$W_{14}^{new} = W_{14} - \eta \delta_4 X_1$$

$$W_{24}^{new} = W_{24} - \eta \delta_4 X_2$$

$$b_4^{new} = b_4 - \eta \delta_4$$

The hidden layer update depends on errors coming back from all output classes.

Complete Numerical Example: Setup

Suppose:

$$X_1 = 0.5, \quad X_2 = 0.75$$

Hidden layer weights:

$$W_{13} = -2.5, \quad W_{23} = 0.6, \quad b_3 = 0.4$$

$$W_{14} = -1.5, \quad W_{24} = 0.4, \quad b_4 = 0.75$$

Output layer weights:

$$W_{35} = -0.1, \quad W_{45} = 1.5, \quad b_5 = 1$$

$$W_{36} = 2.4, \quad W_{46} = -5.2, \quad b_6 = 0.5$$

$$W_{37} = -2.2, \quad W_{47} = 3.7, \quad b_7 = 1$$

Learning rate:

$$\eta = 0.1$$

Target for ReLU-output example:

$$Ob_5 = 0, \quad Ob_6 = 1, \quad Ob_7 = 0$$

Example Step 1: Hidden Forward Pass

For node 3:

$$z_3 = W_{13}X_1 + W_{23}X_2 + b_3$$

$$z_3 = (-2.5)(0.5) + (0.6)(0.75) + 0.4$$

$$= -1.25 + 0.45 + 0.4 = -0.4$$

$$Y_3 = \text{ReLU}(-0.4) = 0$$

For node 4:

$$z_4 = W_{14}X_1 + W_{24}X_2 + b_4$$

$$z_4 = (-1.5)(0.5) + (0.4)(0.75) + 0.75$$

$$= -0.75 + 0.30 + 0.75 = 0.30$$

$$Y_4 = \text{ReLU}(0.30) = 0.30$$

$$Y_3 = 0, \quad Y_4 = 0.30$$

Example Step 2: Output Forward Pass

For node 5:

$$z_5 = W_{35} Y_3 + W_{45} Y_4 + b_5$$

$$\begin{aligned} z_5 &= (-0.1)(0) + (1.5)(0.30) + 1 \\ &= 1.45 \end{aligned}$$

$$Y_5 = \text{ReLU}(1.45) = 1.45$$

For node 6:

$$\begin{aligned} z_6 &= (2.4)(0) + (-5.2)(0.30) + 0.5 \\ &= -1.06 \end{aligned}$$

$$Y_6 = \text{ReLU}(-1.06) = 0$$

For node 7:

$$\begin{aligned} z_7 &= (-2.2)(0) + (3.7)(0.30) + 1 \\ &= 2.11 \end{aligned}$$

$$Y_7 = \text{ReLU}(2.11) = 2.11$$

Example Step 3: Output Errors

Observed outputs:

$$Ob_5 = 0, \quad Ob_6 = 1, \quad Ob_7 = 0$$

Predicted outputs:

$$Y_5 = 1.45, \quad Y_6 = 0, \quad Y_7 = 2.11$$

Errors:

$$Err_5 = Ob_5 - Y_5 = 0 - 1.45 = -1.45$$

$$Err_6 = Ob_6 - Y_6 = 1 - 0 = 1$$

$$Err_7 = Ob_7 - Y_7 = 0 - 2.11 = -2.11$$

ReLU derivatives:

$$\text{ReLU}'(z_5) = 1$$

$$\text{ReLU}'(z_6) = 0$$

$$\text{ReLU}'(z_7) = 1$$

Example Step 4: Output Deltas

Recall:

$$\delta_j = Err_j \text{ReLU}'(z_j)$$

For node 5:

$$\delta_5 = (-1.45)(1) = -1.45$$

For node 6:

$$\delta_6 = (1)(0) = 0$$

For node 7:

$$\delta_7 = (-2.11)(1) = -2.11$$

$\delta_5 = -1.45, \quad \delta_6 = 0, \quad \delta_7 = -2.11$
--

Interpretation:

$$\delta_6 = 0$$

because output node 6 has negative pre-activation, so ReLU blocks the gradient.

Example Step 5: Update Output Weight W_{45}

Use:

$$W_{45}^{new} = W_{45} + \eta Y_4 \delta_5$$

Given:

$$W_{45} = 1.5, \quad \eta = 0.1, \quad Y_4 = 0.30, \quad \delta_5 = -1.45$$

So:

$$W_{45}^{new} = 1.5 + 0.1(0.30)(-1.45)$$

$$= 1.5 - 0.0435$$

$$W_{45}^{new} = 1.4565$$

Because δ_5 is negative, this weight decreases.

Example Step 6: Update Output Bias b_5

Use:

$$b_5^{new} = b_5 + \eta \delta_5$$

Given:

$$b_5 = 1, \quad \eta = 0.1, \quad \delta_5 = -1.45$$

$$b_5^{new} = 1 + 0.1(-1.45)$$

$$= 1 - 0.145$$

$$b_5^{new} = 0.855$$

Same logic applies to other output weights and biases.

Example Step 7: Hidden Deltas

For hidden node 3:

$$\delta_3 = (\delta_5 W_{35} + \delta_6 W_{36} + \delta_7 W_{37}) \text{ReLU}'(z_3)$$

Since:

$$z_3 = -0.4$$

$$\text{ReLU}'(z_3) = 0$$

Therefore:

$$\boxed{\delta_3 = 0}$$

For hidden node 4:

$$\delta_4 = (\delta_5 W_{45} + \delta_6 W_{46} + \delta_7 W_{47}) \text{ReLU}'(z_4)$$

Since:

$$z_4 = 0.30$$

$$\text{ReLU}'(z_4) = 1$$

$$\delta_4 = (-1.45)(1.5) + 0(-5.2) + (-2.11)(3.7)$$

$$= -2.175 - 7.807 = -9.982$$

Example Step 8: Update Hidden Weight W_{24}

Use:

$$W_{24}^{new} = W_{24} + \eta X_2 \delta_4$$

Given:

$$W_{24} = 0.4, \quad \eta = 0.1, \quad X_2 = 0.75, \quad \delta_4 = -9.982$$

$$W_{24}^{new} = 0.4 + 0.1(0.75)(-9.982)$$

$$= 0.4 - 0.74865$$

$$W_{24}^{new} = -0.34865$$

This shows how error from the output layer reaches an earlier hidden-layer weight.

ReLU Output: Full Update Pattern

Output layer:

$$W_{ij}^{new} = W_{ij} + \eta(\text{input to weight})\delta_j$$

Hidden layer:

$$W_{ij}^{new} = W_{ij} + \eta(\text{input to weight})\delta_j$$

The only difference is how δ is calculated.

For output node:

$$\delta_j = Err_j \text{ReLU}'(z_j)$$

For hidden node:

$$\delta_j = \left(\sum_{\text{next nodes } k} \delta_k W_{jk} \right) \text{ReLU}'(z_j)$$

Backpropagation sends error backward layer by layer.

Softmax Example: Correct Class is 5

Suppose output logits are:

$$Y_5, \quad Y_6, \quad Y_7$$

Softmax probabilities are:

$$P_5, \quad P_6, \quad P_7$$

Correct class:

Class 5

Target vector:

$$t = [1, 0, 0]$$

Loss:

$$Loss = -\log(P_5)$$

Gradients:

$$\frac{\partial Loss}{\partial Y_5} = P_5 - 1$$

$$\frac{\partial Loss}{\partial Y_6} = P_6$$

$$\frac{\partial Loss}{\partial Y_7} = P_7$$

Softmax Output Update: Interpretation

For correct class 5:

$$W_{35}^{new} = W_{35} - \eta Y_3 (P_5 - 1)$$

Since:

$$P_5 - 1 < 0$$

we get:

$$-\eta Y_3 (P_5 - 1) > 0$$

So W_{35} increases if $Y_3 > 0$.

Meaning:

The model increases weights that support the correct class.

For wrong class 6:

$$W_{36}^{new} = W_{36} - \eta Y_3 P_6$$

Since $P_6 > 0$, this decreases the support for class 6.

Wrong class weights are pushed down.

- Forward pass calculates hidden outputs and final outputs.
- Loss measures prediction error.
- Backpropagation starts from the output layer.
- Output layer gradients are calculated first.
- Hidden layer receives weighted error from all connected output nodes.
- ReLU derivative can block gradients if pre-activation is negative.
- For softmax with cross-entropy:

$$\frac{\partial \text{Loss}}{\partial Y_j} = P_j - t_j$$

- Gradient descent updates every parameter using:

$$\theta^{new} = \theta - \eta \frac{\partial \text{Loss}}{\partial \theta}$$

Backpropagation is repeated chain rule plus gradient descent.